Silent Node: Antigravity Prompt Map

Staged prompts for building the 4-week district dengue risk forecaster in Google Antigravity

How to use this map

- Work in order. Give the agent **one prompt at a time**, wait for it to finish, review, commit to git, then move on.
- Run Antigravity **on your own machine** with internet access. The pipeline needs Open-Meteo, GitHub and the WHO dashboard.
- Keep terminal approvals **on** for the first few stages so you can see what gets installed and run.
- After every stage, paste the **Review gate** prompt (near the end). Do not move on until the numbers it prints look sensible.
- Choose the strongest model your install offers. The model matters more than the tool.
- If the agent changes one of your decisions, tell it to revert. Decisions live in the brief (Prompt 0).

Decisions to make before you start

Put your answers into Prompt 0 before pasting it.

1. **Kalmunai:** it is a health zone, not an OCHA district polygon. Decide how it maps to Ampara for weather, population and spatial weights, and whether its cases stay a separate series.
2. **GAM tooling:** Python only (`statsmodels` GLM with spline terms) or allow R `mgcv` for the NB GAMs. Python only is simpler to deploy.
3. **Test window:** 2022 to 2026 includes the 2023 outbreak, so stop describing 2023 as training-only.
4. **Risk-tier cutoffs:** let the backtest propose them, but you approve them.
5. **Success definition:** write down the pass/fail rule (beats seasonal-naive on WIS at 2-week horizon, else fall back to seasonal-autoregressive).

Prompt 0: Project brief and ground rules

```
You are building "Silent Node", a weekly-updated, 4-week-ahead dengue risk forecasting system for Sri Lanka's 26 reporting units (25 districts plus the Kalmunai health zone). Users are district vector-control officers / MOH officers who plan field teams 2 to 4 weeks ahead. It is a planning decision-support tool, NOT a clinical tool.

First, create two files in the repo root:
1. docs/PROJECT_BRIEF.md containing the brief below, verbatim.
2. AGENTS.md containing the ground rules below, so they apply to every later task.

BRIEF
- Cases: R package denguedatahub (Talagala), object srilanka_weekly_data, weekly district counts, 2006-W52 to 2026-W20.
- Weather: Open-Meteo ERA5 archive + 14-day forecast at district centroids; CHIRPS as rainfall fallback.
- Climate indices: NOAA ONI and BoM DMI, lagged 3-6 months (tested by ablation only).
- Static: OCHA COD-PS 2023 population (exposure offset, incidence per 100k), CPH-2024 piped-water share, OCHA COD-AB boundaries for queen-contiguity weights.
- Models: negative-binomial GAM per district with distributed lags + LightGBM ensemble. AR terms 1-6 weeks, precipitation lags 4-12 weeks, temperature/humidity lags 8-14 weeks, harmonic seasonality, neighbour-weighted cases at 1-4 week lags.
- Outputs per district: 1-4 week predictive distribution, 50%/95% intervals, probability of exceeding that district's historical 95th-percentile weekly threshold, risk tier (Low/Moderate/High/Extreme), or 'Insufficient Data'.
- Evaluation: rolling-origin backtest 2022-2026, WIS and ROC-AUC, against seasonal-naive, last-value and autoregressive baselines.
- Fallback rule: if climate models do not beat seasonal-naive on WIS at the 2-week horizon, ship the calibrated seasonal-autoregressive model.
- Stack: Python (pandas, statsmodels, scikit-learn, LightGBM), DuckDB, GeoPandas, Streamlit, GitHub Actions.
- My decisions: [PASTE YOUR ANSWERS TO THE 5 DECISIONS HERE]

GROUND RULES (AGENTS.md)
- Never use data that would not have been available at the forecast origin. Treat the most recent 2 weeks of case data as incomplete. Leakage is the top bug to avoid.
- Never invent, interpolate silently, or hard-code data. Every number in a
```

```
report or chart must be computed by code from files in the repo.
- Missing or stale inputs are never defaulted to low risk. They produce
'Insufficient Data'.
- Cache raw downloads in data/raw (git-ignored), cleaned data in
data/processed, and log every download with URL and timestamp.
- Pin dependency versions, set random seeds, write pytest tests for each
module.
- Small modules, type hints, docstrings, one commit per task.
- After each task: stop, summarize what you did, list anything uncertain, and
wait for me.

Do not write any other code yet.
```

Prompt 1: Repo scaffold and case-data ingestion

```
Read docs/PROJECT_BRIEF.md and AGENTS.md.

Task: set up the project and ingest the case data.
1. Create this structure:
src/silentnode/{ingest,features,models,eval,risk,app}, tests/,
data/{raw,processed}, scripts/, docs/. Add pyproject.toml (or
requirements.txt) with pinned versions, .gitignore, and a Makefile with
targets: setup, test, ingest, backtest, forecast, app.
2. Download srilanka_weekly_data from the denguedatahub GitHub repo into
data/raw (log URL + timestamp). Read the .rda into pandas (pyreadr, or R via
a small script if needed).
3. Standardize to a tidy table: district_id, district_name, year, epi_week,
week_start_date, cases. Normalize district names to one canonical list of the
26 units and write the mapping to data/processed/district_lookup.csv.
4. Write validation checks as tests: 26 units present, no duplicate
(district, week) rows, no negative counts, week continuity, row count =
expected. Report any gaps instead of filling them.
5. Store the cleaned table in DuckDB (data/processed/silentnode.duckdb).

Print: number of rows, date range, rows per district, and any validation
warnings. Plot national weekly cases and confirm the July 2017 peak is about
10,590 cases/week. Do not model anything yet.
```

Prompt 2: Weather and climate-index ingestion

```
Task: build the weather pipeline.
1. Create data/processed/district_centroids.csv (lat/lon per district; use
the OCHA boundaries if present, otherwise ask me for a source before
guessing). Apply my Kalmunai decision from the brief.
2. Fetch daily weather from the Open-Meteo ERA5 archive for each district
centroid from 2010-01-01 to the latest available date: precipitation sum,
mean/min/max temperature, mean relative humidity. Respect rate limits with
retry/backoff and cache every raw response to data/raw/weather/.
3. Aggregate to the same epidemiological weeks used by the case table: weekly
rainfall sum, mean temperature, mean humidity.
4. Implement a CHIRPS rainfall fallback as a function, behind a flag. Do not
enable it by default.
5. Add monthly ONI and DMI as optional tables, joined by month with a
configurable lag of 3 to 6 months. Keep them off by default; they are for
ablation later.
6. Add a function that fetches the 14-day forecast, but note that features
with lags of 4 weeks or more do not need it. Keep it optional.

Tests: no missing weeks per district, plausible value ranges, correct week
alignment with the case table.
Report: coverage per district (first/last date, % missing weeks) and a
Colombo plot of weekly rainfall vs weekly cases.
```

Prompt 3: Static covariates and spatial weights

```
Task: build the static covariates and neighbour structure.
1. Load the COD-PS 2023 population file (lka_admpop_adm2_2023.csv) and
compute district population. Map it to the 26 units using my Kalmunai
decision.
2. Parse the CPH-2024 Table A5 'households by main source of drinking water'
with openpyxl and compute the piped-water share per district. Verify:
national about 38.9%, Colombo about 84.4%. If my parse disagrees, stop and
show me the rows instead of adjusting.
3. Load the OCHA COD-AB district boundaries with GeoPandas and build a row-
standardized queen-contiguity weight matrix. Check for islands with no
neighbours and report them.
4. Save everything to data/processed/static_covariates.parquet and
data/processed/spatial_weights.parquet.

Print: a district table (population, piped-water share, number of neighbours)
and a choropleth of population density or incidence per 100k for 2017 as a
sanity check.
```

Prompt 4: Feature store (leakage-safe)

```
Task: build the feature matrix. This is the most leak-prone step, so be
strict.

Use a DIRECT multi-horizon design: for each forecast origin week t and
horizon h in {1,2,3,4}, the target is cases at t+h and every feature is
computed only from data available at t. Never use values after t.

Features per (district, origin t):
- Autoregressive: log1p of cases at lags 0..5 relative to origin (that is,
weeks t to t-5), plus rolling means.
- Rainfall: weekly sums at lags 4..12 weeks; also provide smoothed versions
(for example 3-week rolling windows) because individual lags are highly
collinear.
- Temperature and humidity: lags 8..14 weeks, same smoothing option.
- Seasonality: harmonic terms (sin/cos, 1 and 2 harmonics) and a
southwest/northeast monsoon phase flag.
- Spatial: neighbour-weighted cases (queen weights) at lags 0..3.
- Static: piped-water share; log(population) kept separately as the exposure
offset.
- Reporting delay: a flag marking origins whose latest weeks are incomplete.

Save to DuckDB as features_h{1..4}. Write a leakage test that perturbs all
data after origin t and asserts the features at t do not change.

Report: feature list with names, row counts per horizon, % missing per
feature, and a correlation heatmap of the lag features for Colombo.
```

Prompt 5: Baselines, rolling backtest and metrics

```
Task: build the evaluation framework BEFORE any real model.
1. Rolling-origin backtest: for each weekly origin from 2022-01 to 2026-W20,
train only on data before the origin (excluding the last 2 incomplete weeks),
forecast h=1..4, and store the results. Make retraining frequency
configurable (for example refit every 4 weeks) to keep runtime reasonable.
2. Metrics: Weighted Interval Score (Bracher et al., 2021) using quantile
levels 0.01, 0.025, 0.05, 0.1 to 0.9 in steps of 0.1, 0.95, 0.975, 0.99;
absolute error of the median; 50% and 95% interval coverage; and ROC-AUC for
exceeding each district's 95th-percentile weekly threshold. Compute that
threshold from the training data only, inside each fold.
3. Baselines, each producing full predictive quantiles: seasonal-naive (same
week in past years with a negative-binomial spread fitted to history), last-
value, and a simple autoregressive negative-binomial model.
4. Unit-test the WIS function against a hand-computed example.
5. Output a results table and a chart: WIS by horizon, with district-level
results available.

Print the baseline scores in a table, aggregated across districts and for
Colombo, Gampaha and Kalutara separately.
```

Prompt 6: Negative-binomial GAM per district

```
Task: implement the primary model and plug it into the backtest.
1. For each district and horizon, fit a negative-binomial regression with log(population) as an offset. Use spline terms for seasonality and for the smoothed climate lag features, plus the AR, spatial and static features. Use statsmodels GLM with patsy splines (or R mgcv if my brief allows). Estimate the dispersion parameter, and handle non-convergence explicitly.
2. Each district has only about 1,000 weeks, and lag features are collinear, so apply regularization (penalized fits, or restrict to the smoothed lag summaries). Also fit a pooled model with district effects as a comparison and tell me which is better.
3. Produce predictive quantiles from the fitted NB distribution, including parameter uncertainty if feasible, otherwise state clearly that it is conditional on estimated parameters.
4. Run the full rolling backtest and compare WIS and ROC-AUC against the baselines from Prompt 5, per horizon.

Do not tune on the test window. If you tune anything, use a validation period earlier than 2022 and tell me exactly which one.

Report an honest result: where the model wins, where it loses, and by how much. Do not hide losses.
```

Prompt 7: LightGBM and ensemble

```
Task: add the gradient-boosting model and an ensemble.
1. Train LightGBM with quantile objectives (same quantile levels as the WIS) per horizon, pooled across districts with district as a feature. Use time-ordered splits only.
2. Build an ensemble that combines the NB-GAM quantiles with the LightGBM quantiles (start with a simple average of quantiles, then try weights chosen on a pre-2022 validation period). Prevent crossing quantiles.
3. Backtest with the same framework and add to the comparison table.
4. Check calibration: the 50% and 95% intervals should cover roughly 50% and 95% of outcomes. Show a coverage plot per horizon.

If the ensemble is not better than the best single model, say so and recommend keeping the simpler option.
```

Prompt 8: Ablations and model selection

```
Task: test which inputs earn their place, then apply the fallback rule from
the brief.
1. Ablations, each run through the full backtest: (a) no climate features,
(b) rainfall only, (c) rainfall + temperature/humidity, (d) add ONI/DMI at 3-
6 month lags, (e) no spatial neighbour features, (f) no static covariates,
(g) with vs without the water-access feature.
2. Produce one table: WIS and ROC-AUC per horizon for every variant and
baseline, with a note on whether differences are larger than noise (use a
bootstrap over weeks or a Diebold-Mariano style comparison).
3. Apply the fallback rule exactly as written in the brief: if climate-driven
models do not beat seasonal-naive on WIS at the 2-week horizon, the
operational model becomes the calibrated seasonal-autoregressive model.
4. Write results to docs/EVALUATION.md with tables and charts generated by
code.

Do not change the rule or the metrics to get a better outcome. Report the
result even if it is unflattering.
```

Prompt 9: Risk tiers, exceedance probabilities, Insufficient Data

```
Task: turn forecasts into decision-ready output.
1. For each district and horizon, compute: predicted median and mean cases, incidence per 100,000, 50% and 95% intervals, and probability of exceeding the district's historical 95th-percentile weekly threshold (from the NB distribution or the ensemble quantiles).
2. Propose risk-tier cutoffs (Low / Moderate / High / Extreme) based on exceedance probability and incidence per 100k. Choose cutoffs using the backtest (for example, tier-level hit rates), show me the trade-off table, and wait for my approval before fixing them.
3. Add the 'Insufficient Data' tier. It triggers when a district's recent case data or weather inputs are missing or stale beyond a configurable limit, or when the model failed to fit. It must never default to Low.
4. Add an 'early anomaly flag' for districts whose forecast is well above their seasonal norm while current incidence is still low.
5. Output a single tidy forecast table (CSV and DuckDB) with one row per district per horizon, and a CSV 'allocation matrix' ranked by risk for the next 4 weeks.

Test: a district with deliberately blanked recent data must come out as Insufficient Data.
```

Prompt 10: Forecast engine (one command)

```
Task: build the production run.
Create scripts/run_forecast.py and make forecast do the following in order:
refresh weather, load the latest case data, rebuild features for the latest
valid origin (respecting the 2-week incomplete window), refit the selected
models from Prompt 8, generate forecasts, apply the tiers from Prompt 9, and
write outputs to outputs/latest/ plus a dated copy.

Requirements:
- Idempotent and safe to re-run.
- Structured logging, and a run_metadata.json with the data cut-off week,
model version, and any warnings.
- If a data source is stale, fall back to the last valid period, widen
predictive variance by a documented factor, and set a staleness flag in the
metadata.
- A --as-of flag to reproduce a past forecast from historical data for
testing.

Demo: run with --as-of 2017-05-01 and show what the system would have said
for Colombo before the July 2017 peak.
```

Prompt 11: Live case-data ingestion with fallback

Before this prompt, download 3 to 5 recent weekly PDFs from epid.gov.lk into `data/raw/sample_pdfs/` yourself. The agent must see real files.

```
Task: add live weekly case ingestion.
1. Look at the sample PDFs in data/raw/sample_pdfs/. Describe their table
layout and any variations between weeks before writing code.
2. Write a pdfplumber parser that extracts weekly dengue cases by district,
maps names through district_lookup.csv, and validates the result (26 units,
plausible counts, week number matches the file). Reject and log anything that
fails validation instead of guessing.
3. Add a secondary path using the WHO SEARO dengue dashboard as a fallback.
If you cannot access it programmatically, tell me what you found rather than
faking it.
4. Combine into one ingestion function: try primary, then fallback, then last
valid data with a staleness flag. Record the source and timestamp of each
week's data.
5. Tests using the sample PDFs, plus a test with a deliberately corrupted
PDF.

Report which weeks parsed correctly and which did not.
```

Prompt 12: Streamlit dashboard

```
Task: build the officer-facing dashboard in src/silentnode/app.
Read from outputs/latest/ only; it must not retrain anything.

Layout:
- Top: data-freshness indicator (data cut-off week, last run time) and a
prominent staleness banner when flagged. Permanent non-clinical disclaimer:
this is a planning decision-support tool, not a diagnostic tool.
- Left: choropleth of the 26 units coloured by the selected horizon's risk
tier, with Insufficient Data in a clearly distinct grey/hatched colour.
Horizon selector (1 to 4 weeks).
- Right: ranked table with predicted cases, incidence per 100k, 50% and 95%
intervals, exceedance probability, tier and anomaly flag.
- District detail view: history plus the 4-week fan chart (50% and 95%
bands).
- One-click CSV export of the allocation matrix.
- A 'How to read this' panel explaining exceedance probability and tiers in
plain language for non-statisticians.

Use colour-blind-safe tier colours and don't rely on colour alone (add
labels). Make it run with `make app`. Include a screenshot test or at minimum
a smoke test that the app starts with sample outputs.

Show me screenshots in the browser panel when done.
```

Prompt 13: Automation with GitHub Actions

```
Task: automate the weekly run.
1. Create .github/workflows/weekly_forecast.yml: scheduled weekly (aligned to Epidemiology Unit release timing, configurable), plus manual dispatch. Steps: install, run tests, run forecast, upload outputs as artifacts, and commit or publish the latest outputs to a data branch or release asset.
2. Add a second workflow for tests on pull requests.
3. Cache dependencies and raw downloads sensibly, and keep runtime within free-tier limits. Tell me the measured runtime.
4. If ingestion fails, the workflow must still succeed with a staleness-flagged output, and it must open or log a clear warning rather than silently skipping.
5. Document any repository secrets or settings I must configure by hand. Do not invent secrets.

Explain how to deploy the Streamlit app from the repo (for example Streamlit Community Cloud) in docs/DEPLOY.md.
```

Prompt 14: Verification, documentation and demo

```
Task: final verification and write-up.
1. Run the full test suite and a clean-clone reproduction: fresh virtual environment, `make setup ingest backtest forecast`. Report anything that breaks.
2. Generate docs/EVALUATION.md (final), docs/ARCHITECTURE.md (four stages with a diagram), docs/DATA.md (every source, access date, licence), docs/LIMITATIONS.md, and a README with quick start.
3. In LIMITATIONS.md, be honest about: weak climate correlations, reporting delays, district-level (not MOH-level) resolution, census-snapshot covariates, and the Kalmunai decision.
4. List every number quoted in the proposal (for example 692 weeks, correlation values, case totals) and check each against the data, flagging mismatches.
5. Produce a short demo script (what to click, in order) and capture screenshots for the submission.

Do not edit results to make them look better. Report discrepancies.
```

Review gate (paste after every stage)

```
Review what you just built as a sceptical second engineer.
1. List every assumption you made that I did not specify.
2. Check for data leakage: could any feature or threshold use information
from after the forecast origin?
3. Run the tests and show the output.
4. Show 3 concrete numbers or plots I can sanity-check against known facts
(for example the 2017 peak, Colombo's July seasonality).
5. List anything you could not verify, and anything you would change.
Do not fix anything yet. Just report.
```

Fix prompt template (when something breaks)

```
Problem: [what you saw: error text, wrong number, bad chart]
Expected: [what you thought should happen]
First, explain the likely cause in plain language. Then show me the minimal
fix, apply it, rerun the relevant test, and show the evidence it is fixed. Do
not refactor unrelated code.
```

If time is short

If the timeline is tight, keep this order and cut from the bottom: Prompts 0, 1, 2, 3, 4, 5, 6, 9, 10, 12 are the core. Prompts 7 and 8 (ensemble and ablations) matter for credibility, but the fallback rule means a clean seasonal-autoregressive plus NB-GAM system with honest backtests is still a complete submission. Prompts 11 and 13 (live PDFs and automation) can be demonstrated with the last historical week if needed, as long as you say so plainly.